Phase IV: Software Testing

Project: PawMatch — Pet Adoption Platform

1. Introduction to Testing

Software testing is the systematic process of evaluating a software product to identify defects, bugs, security flaws, or any behaviour that deviates from the documented requirements. Testing is performed by writing controlled inputs and comparing the actual output of the system to the expected output. When the two disagree, a defect has been found.

Testing is a critical activity in modern software development for several reasons:

- **Reliability** — automated tests can be re-run after every change, catching regressions long before users do.
- **Correctness** — tests document and enforce the precise behaviour expected from each function, eliminating ambiguity.
- **Maintainability** — well-tested code can be refactored fearlessly because the test suite verifies that the external behaviour is preserved.
- **Security** — for components such as authentication, tests verify that invalid inputs cannot bypass protections.
- **Confidence** — a passing test suite gives the team confidence to ship new features quickly.

In PawMatch, testing is essential because the platform handles user accounts, JWT-protected resources, and role-based access. A single broken endpoint could expose private data or break the adoption flow for every user of the platform.

2. Purpose of Testing

The purpose of testing in PawMatch is to:

1. **Detect defects early.** A bug caught while a test fails on the developer's laptop costs minutes to fix; the same bug discovered by users in production may cause data loss, downtime, or security incidents.
2. **Verify behaviour under expected conditions.** Tests confirm that the happy paths (valid login, valid pet listing, valid booking) produce the correct response codes, payloads, and side effects in the database.
3. **Verify behaviour under unexpected conditions.** Tests deliberately submit invalid, empty, or malicious input to confirm that the system rejects them with a clear error message instead of

crashing or returning sensitive data.

4. **Prevent regressions.** Once a bug is fixed and covered by a test, that exact failure can never silently return.
5. **Document the contract.** A test is executable documentation: it states exactly what input produces what output, and it stays in sync with the code because if the code changes, the test breaks.

3. Focus on Testing Critical Components

Three critical components were selected from the PawMatch codebase:

Component 1: Login Function (`server/routes/auth.js`)

The **Login Function** is the gateway to every authenticated feature of PawMatch. It controls user access to the system. If the login procedure does not work correctly, legitimate users may be locked out of their accounts, or unauthorized users may gain access to private data such as bookings, donations, and admin panels.

This component is also critical because it must check multiple conditions:

- whether the email exists in the database,
- whether the password matches the stored bcrypt hash,
- whether the request contains empty fields,
- whether the response includes a valid JWT token signed with the correct secret.

Because the login process directly affects **system security and user experience**, it must be tested carefully against valid inputs, invalid inputs, empty inputs, and boundary cases.

Component 2: Pet Operations (`server/routes/pets.js`)

The **Pet Operations** component handles all CRUD (Create, Read, Update, Delete) operations on pet listings — the central entity of the entire platform. It is critical because:

- Public users browse pets without authentication.
- Authenticated shelters create and edit their own pet listings.
- Admins update statuses (available → pending → adopted) and delete records.
- The response shape must be normalized for the frontend (nested `health` and `goodWith` objects, plus a singular `image` field derived from a JSON array).

A bug in this component would either expose the wrong data, break the adoption flow, or allow non-owners to modify a shelter's pets.

Component 3: Authentication Middleware (indirectly tested through Pet Operations)

The **Authentication Middleware** (`server/middleware/auth.js`) is exercised by every test that calls a protected endpoint on Pet Operations. It verifies the JWT signature, decodes the payload into `req.user`, and rejects requests with missing, malformed, or expired tokens. Authorization checks (admin-only, owner-only) are also covered by tests TC14, TC15, and TC19.

4. Preparing Test Cases

Test cases were designed to cover **three categories of scenarios** for each component

- **Normal inputs** — valid credentials, valid pet IDs, valid filters.
- **Invalid inputs** — wrong password, non-existent email, missing token, wrong role.
- **Boundary / edge cases** — empty strings, surrounding whitespace, malformed tokens, non-existent IDs.

Login Function — Test Cases

Test ID	Scenario	Input	Expected Result
TC01	Valid admin login	`admin@pawmatch.com` / `epoka123`	200 OK, JWT token, user object with `role: 'admin'`
TC02	Wrong password	`admin@pawmatch.com` / `wrongpass`	401 Unauthorized, message "Invalid email or password."
TC03	Unknown email	`doesnotexist@nowhere.com` / `anything`	401 Unauthorized
TC04	Empty email	`''` / `epoka123`	400 Bad Request, message contains "required"
TC05	Empty password	`admin@pawmatch.com` / `''`	400 Bad Request, message contains "required"
TC06	Both fields empty	`''` / `''`	400 Bad Request
TC07	Boundary — surrounding spaces	`' admin@pawmatch.com '` / `epoka123`	401 Unauthorized (spaces are not trimmed by design)
TC08	Valid shelter login	`happytails@shelter.com` / `password123`	200 OK, user object with `role: 'shelter'`
TC09	JWT structure validation	Decode token after valid login	Three base64-encoded parts; payload contains correct `role` and `email`

Pet Operations — Test Cases

Test ID	Scenario	Input	Expected Result
TC10	GET all pets	`GET /api/pets`	200 OK, non-empty array
TC11	Response normalization	`GET /api/pets`	Each pet has `image: string`, `health: object`, `goodWith: object`, `shelterId: string`
TC12	GET pet by ID (valid)	`GET /api/pets/3`	200 OK, pet object with matching ID
TC13	GET pet by ID (invalid)	`GET /api/pets/non-existent-id`	404 Not Found, message contains "not found"
TC14	UPDATE without token	`PUT /api/pets/3` (no auth header)	401 Unauthorized
TC15	UPDATE with non-owner user token	`PUT /api/pets/3` (regular user token)	403 Forbidden
TC16	UPDATE with admin token	`PUT /api/pets/3` with `{status: 'pending'}` and admin token	200 OK, pet status updated to `pending`
TC17	Filter by type	`GET /api/pets?type=dog`	200 OK, every result has `type: 'dog'`
TC18	Search by name	`GET /api/pets?search=Bella`	200 OK, results include the searched pet
TC19	Boundary — invalid token	`PUT /api/pets/3` with `Authorization: Bearer not-a-valid-jwt`	401 or 403, no pet returned

---

5. Writing Test Code

The tests were written using **Vitest**, a fast JavaScript testing framework that integrates naturally with the Vite-based PawMatch stack. The tests run as HTTP integration tests against the live backend on `http://localhost:5001`, which means they exercise the **entire** request path: routing, middleware, controllers, database queries, and JSON serialization — exactly as the real frontend does.

Setup — `server/package.json`

```
"scripts": {
  "test": "vitest run --reporter=verbose"
},
"devDependencies": {
  "vitest":  "^4.1.5"
}
```

Test file 1 — `server/tests/auth.test.js`

```
import { describe, test, expect } from 'vitest';

const API = 'http://localhost:5001/api';

async function login(email, password) {
  const res = await fetch(`${API}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password }),
  });
  return { status: res.status, body: await res.json() };
}

describe('Login Function — auth.js POST /api/auth/login', () => {
  test('TC01 — Valid login (admin) returns token and user', async () => {
    const { status, body } = await login('admin@pawmatch.com', 'epoka123');
    expect(status).toBe(200);
    expect(body.message).toBe('Login successful');
    expect(body.token).toBeTypeOf('string');
    expect(body.user.role).toBe('admin');
  });

  test('TC02 — Wrong password returns 401', async () => {
    const { status, body } = await login('admin@pawmatch.com', 'wrongpass');
    expect(status).toBe(401);
    expect(body.message).toBe('Invalid email or password.');
    expect(body.token).toBeUndefined();
  });

  test('TC04 — Empty email returns 400', async () => {
    const { status, body } = await login('', 'epoka123');
    expect(status).toBe(400);
    expect(body.message).toMatch(/required/i);
  });

  test('TC09 — Token can be decoded as JWT (header.payload.signature)', async () =>
{
    const { body } = await login('admin@pawmatch.com', 'epoka123');
    const parts = body.token.split('.');
```

```
    expect(parts).toHaveLength(3);
    const payload = JSON.parse(Buffer.from(parts[1], 'base64').toString());
    expect(payload.role).toBe('admin');
  });
});
```

Test file 2 — `server/tests/pets.test.js`

```
import { describe, test, expect, beforeAll } from 'vitest';

const API = 'http://localhost:5001/api';
let adminToken;
let firstPetId;

beforeAll(async () => {
  const res = await fetch(`${API}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email: 'admin@pawmatch.com', password: 'epoka123' }),
  });
  adminToken = (await res.json()).token;

  const pets = await (await fetch(`${API}/pets`)).json();
  firstPetId = pets[0].id;
});

describe('Pet Operations — pets.js routes', () => {
  test('TC11 — Pet response is normalized (image, health, goodWith)', async () => {
    const pets = await (await fetch(`${API}/pets`)).json();
    const p = pets[0];
    expect(p.image).toBeTypeOf('string');
    expect(p.health.vaccinated).toBeTypeOf('boolean');
    expect(p.goodWith.kids).toBeTypeOf('boolean');
    expect(p.shelterId).toBeTypeOf('string');
  });

  test('TC16 — PUT /api/pets/:id with admin token updates status', async () => {
    const res = await fetch(`${API}/pets/${firstPetId}`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${adminToken}`,
      },
      body: JSON.stringify({ status: 'pending' }),
    });
    expect(res.status).toBe(200);
    const body = await res.json();
    expect(body.pet.status).toBe('pending');
```

```
  });
});
```

Assertions used

Assertion	Purpose
`expect(status).toBe(200)`	Verifies HTTP status code matches exactly.
`expect(body.token).toBeTypeOf('string')`	Verifies the token field is present and is a string.
`expect(body.message).toMatch(/required/i)`	Verifies the error message matches a regular expression.
`expect(Array.isArray(pets)).toBe(true)`	Verifies the response is an array.
`expect([401, 403]).toContain(res.status)`	Verifies status is one of the acceptable values.
`expect(body.token).toBeUndefined()`	Verifies a sensitive field is absent in error responses.

---

6. Running Tests

How the tests were executed

The tests were executed with a single command from the `server/` directory:

```
npm test
```

This command invokes Vitest, which:

1. Discovers all files matching `*.test.js` inside `server/tests/`.
2. Loads each file and registers every `describe` and `test` block.
3. Executes the `beforeAll` setup hooks (logs in as admin and as a regular user).
4. Executes each test in parallel where possible, against the live backend running on port 5001.
5. Collects results and prints a verbose colour-coded report.

Test execution output

```
> pawmatch-server@1.0.0 test
> vitest run --reporter=verbose

 RUN  v4.1.5 /Users/alketaalia/Downloads/pet-adoption-platform-development
1/server
```

```
 √ tests/auth.test.js  (9 tests)
   √ TC01 — Valid login (admin) returns token and user                    119ms
   √ TC02 — Wrong password returns 401                                     99ms
   √ TC03 — Unknown email returns 401                                      1ms
   √ TC04 — Empty email returns 400                                        1ms
   √ TC05 — Empty password returns 400                                     1ms
   √ TC06 — Both fields empty returns 400                                  1ms
   √ TC07 — Boundary: email with surrounding spaces should fail            1ms
   √ TC08 — Valid login (shelter role) returns shelter user               51ms
   √ TC09 — Token can be decoded as JWT (header.payload.signature)        52ms

 √ tests/pets.test.js  (10 tests)
   √ TC10 — GET /api/pets returns a non-empty array                        1ms
   √ TC11 — Pet response is normalized (image, health, goodWith)           1ms
   √ TC12 — GET /api/pets/:id returns the requested pet                    1ms
   √ TC13 — GET /api/pets/:id with invalid ID returns 404                  1ms
   √ TC14 — PUT /api/pets/:id without token returns 401                   51ms
   √ TC15 — PUT /api/pets/:id with non-owner user token returns 403       52ms
   √ TC16 — PUT /api/pets/:id with admin token updates status              4ms
   √ TC17 — Filter by type returns only matching pets                      1ms
   √ TC18 — Search by name returns matching pets                           1ms
   √ TC19 — Invalid token signature is rejected                            1ms

 Test Files  2 passed (2)
      Tests  19 passed (19)
   Start at  14:12:25
   Duration  397ms
```

Interpreting the results

- A green ✓ indicates the test passed — actual output matched the expected output.
- A red ✗ indicates the test failed — Vitest prints the expected value, the received value, and the file and line where the assertion was made.
- A yellow ⚠ would indicate a test that was skipped.

Defect found and fixed during testing

During the first test run, **6 login tests failed** because the original `login` route returned `401 Unauthorized` for empty fields instead of `400 Bad Request`. This was a real defect — the API silently treated empty input the same as wrong credentials, which is misleading for the frontend.

The fix added an explicit guard at the top of the handler:

```
if (!email || !password || email.trim() === '' || password === '') {
  return res.status(400).json({ message: 'Email and password are required.' });
}
```

After the fix, all 19 tests passed. This illustrates the **value of testing**: a missing validation was caught before it could confuse users in production.

---

7. Test Coverage

Why test coverage matters

Test coverage measures how much of the production code is exercised by the test suite. High coverage gives confidence that regressions will be caught; low coverage means large portions of the code are untested and may contain hidden defects. Coverage alone is not sufficient — tests must also assert meaningful behaviour — but it is a useful signal of testing thoroughness.

Coverage analysis for PawMatch

The 19 test cases cover the following paths in the selected components:

Login function (`auth.js → POST /login`)

Path / Condition	Covered by
Valid credentials, admin role	TC01, TC09
Valid credentials, shelter role	TC08
Wrong password (bcrypt mismatch)	TC02
Email not found in DB	TC03
Empty email	TC04
Empty password	TC05
Both fields empty	TC06
Whitespace-padded email	TC07
JWT signature, structure, payload	TC09

All major branches of the login handler are covered: input validation, DB lookup miss, bcrypt mismatch, success path, and token generation.

Pet Operations (`pets.js → GET, GET/:id, PUT/:id`)

Path / Condition	Covered by
GET all pets — happy path	TC10

Path / Condition	Covered by
Response normalization (`image` , `health` , `goodWith`)	TC11
GET pet by ID — found	TC12
GET pet by ID — not found	TC13
PUT — no auth token	TC14
PUT — wrong role / non-owner	TC15
PUT — admin succeeds	TC16
Filter query parameter	TC17
Search query parameter	TC18
Invalid JWT signature	TC19

All authorization branches (no token, wrong role, admin) are covered, as well as the happy path, normalization, and query parameters.

What is still untested

Honestly assessing the limits of this test suite:

- **DELETE endpoints** were not covered — they would need a separate test that creates a temporary pet, deletes it, and verifies it is gone.
- **POST (create pet)** was not tested as a shelter user.
- **Token expiry** could be tested by issuing a token with `expiresIn: '1ms'` and waiting.
- **SQL injection attempts** — the routes use parameterized queries (`?` placeholders), so they should be safe, but a dedicated test could confirm.
- **Concurrent updates** — race conditions are not exercised.
- **Frontend components** (React) are not covered by this suite; a separate frontend test setup (Vitest + React Testing Library) would be needed.

Reflection

The test suite achieves **strong coverage of the most security-critical and business-critical paths**: authentication and pet status management. It found and fixed one real defect (missing validation for empty login fields). The tests are fast (under 0.5 seconds for the entire suite) and reproducible, which means they can be run on every commit before merging changes.

For a production deployment, the next steps would be:

1. Add tests for the remaining endpoints (bookings, reviews, donations, events, notifications).
2. Add unit tests for the helper functions (e.g., the `normalizePet` mapper).
3. Add frontend component tests with React Testing Library.

4. Integrate the test suite with a CI pipeline (e.g., GitHub Actions) so tests run automatically on every pull request.
5. Measure code coverage with `vitest --coverage` and target ≥80% line coverage for critical files.